

PROJECT REPORT

HOSPITAL MANAGEMENT SYSTEM

Project Title	Hospital Management System
Technology Used	Core Java (JDK 14+)
Submitted By	CHITRAKSHI CHUAHAN
Registration No.	25BAI11264
Course	PROGRAMING IN JAVA
Department	SCAI
Institution	VIT BHOPAL
Guide / Faculty	DR. M. SURESH
Date of Submission	18 SEPTEMBER 2026

Table of Contents

1. Abstract	3
2. Introduction	3
3. Objectives	3
4. System Requirements	4
5. System Design.....	5
6. Object-Oriented Concepts Applied	7
7. Exception Handling	7
8. File Handling and Data Persistence	9
9. Modules and Functionality.....	9
10. Compilation and Execution	11
11. Sample Output.....	11
12. Testing	13
13. Limitations	13
14. Future Enhancements	13
15. Conclusion	14
16. References	14

1. Abstract

The Hospital Management System is a menu-driven console application developed entirely in core Java, without the use of any external library, framework or database. The project automates the routine record-keeping activities of a small hospital or clinic: registering doctors and patients, booking and cancelling appointments, updating the status of an appointment, and viewing consolidated records of all three entities.

The primary academic purpose of the project is to demonstrate, in a single cohesive application, three core areas of the Java language: object-oriented programming with inheritance and abstraction, a custom checked-exception hierarchy for domain-specific error handling, and file handling for data persistence. All records entered during a session are written to plain text files, so that data survives between program runs without requiring a database engine.

2. Introduction

Hospitals handle a continuous flow of information — patient details, doctor availability, and the appointments that link the two. When these records are maintained manually in registers, the process becomes slow, prone to duplication, and difficult to search. A simple software system can remove most of this effort.

This project implements such a system at a scale appropriate for an academic exercise. It runs in a terminal, presents a numbered menu, and stores its data in three comma-separated text files. Although it is deliberately small, the design follows the same principles a larger system would use: a common abstract base class for shared attributes, specialised subclasses for each entity type, a dedicated class for persistence, and a well-defined set of exceptions raised whenever the user supplies data that the system cannot accept.

2.1 Problem Statement

To design and implement a console-based Hospital Management System in core Java that allows hospital staff to register doctors and patients, book and manage appointments, prevent scheduling conflicts, report invalid input

through meaningful error messages rather than crashing, and retain all records permanently in text files.

2.2 Scope of the Project

- Registration of doctors with their specialization and available consultation slots.
- Registration of patients with their disease or symptoms and residential address.
- Booking of an appointment between a registered patient and a registered doctor on a specified date and time.
- Cancellation of an existing appointment, and updating an appointment status to COMPLETED or CANCELLED.
- Listing of all patients, all doctors and all appointments currently in the system.
- Automatic saving of all records to text files on exit, and automatic reloading on the next run.

3. Objectives

- To apply object-oriented design by modelling real-world entities (Person, Doctor, Patient, Appointment) as Java classes.
- To demonstrate inheritance and abstraction through an abstract Person superclass extended by Doctor and Patient.
- To demonstrate runtime polymorphism by overriding a display method in each subclass.
- To design and use a hierarchy of user-defined checked exceptions for domain-specific error conditions.
- To implement file handling using the java.io package so that application data persists across sessions.
- To build a robust, menu-driven user interface that validates input and never terminates abnormally on incorrect entry.

4. System Requirements

4.1 Software Requirements

Requirement	Specification
Operating System	Windows 10/11, macOS, or any Linux distribution
Language	Core Java (Java SE)
JDK Version	JDK 14 or later (JDK 17 / 21 LTS recommended)
Packages Used	java.util (Scanner, ArrayList), java.io (File, BufferedReader, BufferedWriter, FileReader, FileWriter)
External Libraries	None — no Maven, Gradle or npm dependency
Database	None — plain .txt files are used for storage
Editor / IDE	Any text editor, or IntelliJ IDEA / Eclipse / VS Code (optional)

4.2 Hardware Requirements

Component	Minimum Specification
Processor	Any dual-core processor, 1.5 GHz or above
RAM	2 GB (4 GB recommended)
Disk Space	Approximately 500 MB for the JDK; the project itself needs under 1 MB
Display	Standard monitor with terminal/console access
Network	Not required — the application runs fully offline

5. System Design

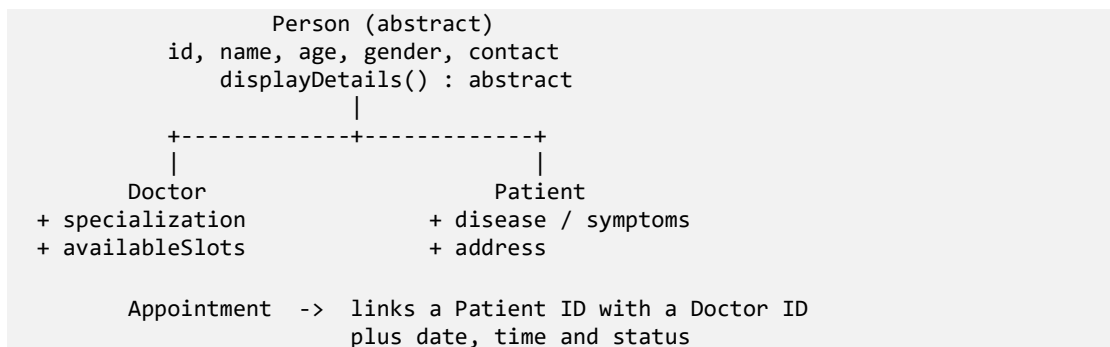
5.1 Architecture Overview

The application follows a simple three-layer separation of responsibilities:

- Model layer — Person, Doctor, Patient and Appointment hold the data and expose it through getters and setters.
- Persistence layer — FileManager is solely responsible for reading records from and writing records to the text files.
- Presentation / control layer — HospitalManagementSystem contains the main method, displays the menu, reads user input, invokes the appropriate operation, and catches the exceptions raised by the lower layers.

Because the model classes know nothing about the console and the persistence class knows nothing about the menu, any one layer can be modified without disturbing the others. Replacing the text files with a database, for example, would require changes only inside FileManager.

5.2 Class Hierarchy



5.3 Description of Classes

Class / File	Type	Responsibility
Person.java	Abstract class	Base class holding the attributes common to every person in the system: id, name, age, gender and contact. Declares an abstract display method that every subclass must implement.
Doctor.java	Concrete subclass	Extends Person. Adds specialization and the list of available consultation slots. Overrides the display method.

Class / File	Type	Responsibility
Patient.java	Concrete subclass	Extends Person. Adds the disease or reported symptoms and the residential address. Overrides the display method.
Appointment.java	Entity class	Represents a single booking. Stores the appointment ID, patient ID, doctor ID, date, time and current status (BOOKED, COMPLETED or CANCELLED).
HospitalException.java	Custom exception	Base checked exception from which all other application exceptions are derived.
InvalidAppointmentException.java	Custom exception	Raised when the date, time or status supplied by the user is not in a valid format or is not a recognised value.
DoctorUnavailableException.java	Custom exception	Raised when the selected doctor already has an appointment at the requested date and time.
RecordNotFoundException.java	Custom exception	Raised when a patient, doctor or appointment ID entered by the user does not exist in the system.
FileManager.java	Utility class	Handles all input and output with patients.txt, doctors.txt and appointments.txt.
HospitalManagementSystem.java	Main class	Contains main(). Displays the menu, accepts user choices, calls the appropriate operations, and handles exceptions.

6. Object-Oriented Concepts Applied

6.1 Abstraction

Person is declared abstract because a "person" in isolation is never created in this system — only a doctor or a patient is. The class captures what is common to both and declares an abstract `displayDetails()` method, forcing every subclass to define how its own details are printed while hiding those implementation details from the calling code.

6.2 Inheritance

Doctor and Patient both extend Person and therefore inherit the `id`, `name`, `age`, `gender` and `contact` fields together with their accessor methods. Each subclass then adds only the attributes that are unique to it. This removes duplicated code and creates a natural "is-a" relationship: a Doctor is a Person, and a Patient is a Person.

```
public abstract class Person {
    protected String id, name, gender, contact;
    protected int age;
    public abstract void displayDetails();
}

public class Doctor extends Person {
    private String specialization;
    private String availableSlots;
    @Override
    public void displayDetails() { /* doctor-specific output */ }
}
```

6.3 Polymorphism

`displayDetails()` is overridden in both Doctor and Patient. When a collection of Person references is traversed, the version that executes is decided at runtime from the actual object type — this is method overriding, or runtime polymorphism. Constructor overloading is also used where objects are created either from user input or from a line read out of a file.

6.4 Encapsulation

All instance variables are declared private (or protected within the base class) and are reached only through public getter and setter methods. External code therefore cannot place an object into an invalid state directly; every change passes through a method that can validate it.

7. Exception Handling

Rather than relying on the generic exceptions provided by the JDK, the project defines its own hierarchy of checked exceptions. Each one carries a message that describes the problem in the language of the domain, so the user sees a meaningful explanation instead of a stack trace.

Exception	Extends	Raised When
HospitalException	Exception	Base class — never thrown directly; allows a single catch block to handle any application error.
RecordNotFoundException	HospitalException	A patient, doctor or appointment ID entered by the user does not exist.
DoctorUnavailableException	HospitalException	The chosen doctor is already booked for the same date and time.
InvalidAppointmentException	HospitalException	The date, time or status value supplied is invalid or badly formatted.

Because all three specific exceptions inherit from `HospitalException`, the menu loop can catch the base type once and still print the precise message of whichever subclass was actually thrown:

```
try {
    bookAppointment(patientId, doctorId, date, time);
} catch (HospitalException e) {
    System.out.println("Error: " + e.getMessage());
}
```

In addition, `InputMismatchException` and `NumberFormatException` from the standard library are caught wherever numeric input is read, and `IOException` is handled inside `FileManager`. The result is an application that reports every error politely and returns to the menu instead of terminating.

8. File Handling and Data Persistence

The project uses the `java.io` package to achieve persistence. When the user chooses "Save & Exit", `FileManager` writes every record currently held in memory to three plain text files created in the directory from which the program was launched. On the next run, the same class reads those files back and repopulates the in-memory lists, so the hospital records continue exactly where they were left.

File	Contents	Record Format
patients.txt	One line per registered patient	id, name, age, gender, contact, disease, address
doctors.txt	One line per registered doctor	id, name, age, gender, contact, specialization, availableSlots
appointments.txt	One line per appointment	appointmentId, patientId, doctorId, date, time, status

Writing is performed with `FileWriter` wrapped in a `BufferedWriter`; reading uses `FileReader` wrapped in a `BufferedReader`. Each line is split on the comma delimiter with `String.split(",")` and the resulting array is passed to the appropriate constructor. If a file does not yet exist — which is the case on the very first run — the read operation is skipped silently and the system simply starts with empty lists.

```
// Writing
BufferedWriter bw = new BufferedWriter(new FileWriter("doctors.txt"));
for (Doctor d : doctors) {
    bw.write(d.getId() + "," + d.getName() + "," + d.getAge() + ",");
    bw.write(d.getGender() + "," + d.getContact() + ",");
    bw.write(d.getSpecialization() + "," + d.getAvailableSlots());
    bw.newLine();
}
bw.close();

// Reading
BufferedReader br = new BufferedReader(new FileReader("doctors.txt"));
String line;
while ((line = br.readLine()) != null) {
    String[] f = line.split(",");
    doctors.add(new Doctor(f[0], f[1], Integer.parseInt(f[2]),
                           f[3], f[4], f[5], f[6]));
}
br.close();
```

9. Modules and Functionality

Option	Module	Description
1	Add Doctor	Registers a new doctor with ID, name, age, gender, contact, specialization and available slots.
2	Add Patient	Registers a new patient with ID, name, age, gender, contact, disease/symptoms and address.
3	Book Appointment	Creates an appointment after verifying that both IDs exist and that the doctor is free at that date and time.
4	Cancel Appointment	Cancels an existing appointment identified by its appointment ID.

Option	Module	Description
5	View All Appointments	Displays every appointment with its patient, doctor, date, time and status.
6	View All Patients	Displays the full list of registered patients.
7	View All Doctors	Displays the full list of registered doctors with their specializations.
8	Update Appointment Status	Changes the status of an appointment to COMPLETED or CANCELLED.
9	Save & Exit	Writes all records to the text files and terminates the program.

10. Compilation and Execution

Navigate to the project folder that contains the hospital sub-folder and compile all source files together:

```
cd path/to/HospitalManagementSystem
javac hospital/*.java
```

A successful compilation produces no output and creates .class files inside the hospital folder. Run the application from the same directory:

```
java hospital.HospitalManagementSystem
```

11. Sample Output

Main menu displayed on startup:

```
===== HOSPITAL MANAGEMENT SYSTEM =====
1. Add Doctor
2. Add Patient
3. Book Appointment
4. Cancel Appointment
5. View All Appointments
6. View All Patients
7. View All Doctors
8. Update Appointment Status
9. Save & Exit
Enter choice: 1
```

Registering a doctor:

```
Enter Doctor ID      : D1
Enter Name           : Dr. Arun Mehta
Enter Age            : 45
Enter Gender         : Male
Enter Contact        : 9876543210
Enter Specialization : Cardiology
Enter Available Slots : 10:00-13:00

Doctor added successfully.
```

Booking an appointment:

```
Enter Patient ID : P1
Enter Doctor ID  : D1
Enter Date (dd-mm-yyyy) : 12-10-2026
Enter Time (HH:mm)      : 11:30

Appointment booked successfully. Appointment ID: A1
```

Exception raised when the doctor is already booked for that slot:

```
Enter Patient ID : P2
Enter Doctor ID  : D1
Enter Date (dd-mm-yyyy) : 12-10-2026
Enter Time (HH:mm)      : 11:30

Error: Doctor D1 is already booked at 11:30 on 12-10-2026.
```

Exception raised when an ID does not exist:

Enter Patient ID : P9

Error: No patient found with ID P9.

12. Testing

The application was tested manually against the following cases. In every negative case the system displayed a message and returned to the menu without terminating.

#	Test Case	Expected Result	Status
1	Add a doctor with valid details	Doctor stored and listed under option 7	Pass
2	Add a patient with valid details	Patient stored and listed under option 6	Pass
3	Book an appointment with valid IDs	Appointment created with a generated ID	Pass
4	Book with a non-existent patient ID	RecordNotFoundException message shown	Pass
5	Book the same doctor at the same date and time	DoctorUnavailableException message shown	Pass
6	Enter a date in the wrong format	InvalidAppointmentException message shown	Pass
7	Update status to an unrecognised value	InvalidAppointmentException message shown	Pass
8	Enter text where a number is expected	Input error message shown, menu redisplayed	Pass
9	Save & Exit, then restart the program	All previously entered records reloaded	Pass
10	Run the program for the first time	Starts with empty lists, no file error	Pass

13. Limitations

- Patient and doctor IDs are entered manually and are not auto-generated, so uniqueness depends on the operator.
- Data is stored in plain text files with no encryption, so the records are not secure against direct file access.
- Since fields are separated by commas, a comma typed inside a name or address would corrupt that record.
- The application is single-user and console-based; it has no graphical interface and cannot be accessed concurrently.
- There is no billing, prescription, ward allocation or medical-history module.

14. Future Enhancements

- Replace the text files with a relational database such as MySQL, accessed through JDBC.
- Add a graphical user interface using JavaFX or Swing, or expose the system as a web application.
- Introduce automatic ID generation and login authentication with separate roles for admin, doctor and receptionist.

- Extend the system with billing, prescriptions, laboratory reports and a searchable medical history.
- Use the `java.time` API for strict validation of dates and times, and send appointment reminders by email or SMS.

15. Conclusion

The Hospital Management System successfully meets the objectives set out at the start of the project. It provides a working solution for registering doctors and patients, scheduling appointments without conflicts, and retaining all records between sessions — using nothing beyond the core Java language.

More importantly for its academic purpose, the project brings together several fundamental concepts in one coherent application. Abstraction and inheritance shape the class hierarchy around a common `Person` base class; polymorphism allows each entity to present itself differently; a custom exception hierarchy converts every foreseeable error into a clear message rather than an abnormal termination; and file handling gives the application permanent memory. The modular structure also leaves the system open to extension, so the enhancements listed in the previous section could be added without redesigning what already exists.

16. References

- Herbert Schildt, *Java: The Complete Reference*, McGraw Hill Education.
- Oracle, *Java SE Documentation and API Specification* — docs.oracle.com/en/java/javase
- Oracle, *The Java Tutorials: Inheritance, Exceptions and Basic I/O trails*.
- E. Balagurusamy, *Programming with Java*, McGraw Hill Education.
- Project source code and README of the Hospital Management System (submitted repository).